COMPOSITION, INHERITANCE AND VIRTUAL FUNCTIONS

COMPOSITION

Inheritance and Composition

- The two common ways to relate two classes in a meaningful way are:
  - Inheritance (“is-a” relationship)
  - Composition (“has-a” relationship)

COMPOSITION (OBJECTS INSIDE THE OBJECTS)

Composition

- In composition, one or more member(s) of a class are objects of another class type
- Composition is a “has-a” relation
- Arguments to the constructor of a member-object are specified in the heading part of the definition of the constructor

Objects inside objects (1)

```
class Element {
        int value;
public:
        int Get(void) { return value; }
        void Put(int val) { value = val; }
};
class Collection {
        Element el1, el2;
public:
        int Get(int elno) { return elno == 1 ? el1.Get() : el2.Get(); }
        int Put(int elno, int val) { if(elno == 1) el1.Put(val); else el2.Put(val); }
};
int main(void) {
        Collection coll;
        for(int i = 1; i <= 2; i++)
                coll.Put(i, i + 1);
        for(int i = 1; i <= 2; i++)
                cout << "Element #" << i << " = " << coll.Get(i) << endl;
        return 0;
}
```

Output

```
Element #1 = 2
Element #2 = 3
```

Objects inside objects (1)

An object of any class may be the field of an object of any other class.

All rules concerning access to class components are honoured in this case too.

There's a very simple class named *Element*.

It's intended to store one *int* value.

The second class displayed in the example is also simple, but there's an interesting thing about it: there are two fields of the *Element* class.

As you can see, we can manipulate them with no great difficulty. They behave just like any other class component.

It'll be no surprise to you when we say that the code produces the following output:

```
Element #1 = 2
Element #2 = 3
```

There are **three objects** in our example program:

coll(visible at the *main* function level) and *el1* together with *el2* (visible at the *coll* object level).

We want to know the sequence in which all constructors work.

Objects inside objects (2)

Next, the modified code is shown.

We've cleaned the main function of everything that isn't really needed to track the constructors activity.

We've compiled and run the program. This is what we see on the screen:

Element constructed!
Element constructed!
Collection constructed!

The conclusion is: **constructors from inner objects (objects stored inside other objects) are invoked before the outer object's constructors start their work**.

This rule should be applied repeatedly.

Objects inside objects (2)

```
class Element {
        int value;
public:
        Element(void) { cout << "Element constructed!" << endl; }
        int Get(void) { return value; }
        void Put(int val) { value = val; }
};
class Collection {
        Element el1, el2;
public:
        Collection(void) { cout << "Collection constructed!" << endl; }
        int Get(int elno) { return elno == 1 ? el1.Get() : el2.Get(); }
        int Put(int elno, int val) { if(elno == 1) el1.Put(val); else el2.Put(val); }
};
int main(void) {
        Collection coll;
        return 0;
}
```

Output

```
Element constructed!
Element constructed!
Collection constructed!
```

Objects inside objects (3)

Next, we changed a **parameter-less constructor** to **a one-parameter constructor**.

The modification has invalidated the program. It can't be compiled. Why?
The compiler will produce an error message saying something like this:
In constructor 'Collection::Collection()':
error: no matching function for call to 'Element::Element()'

The constructor invoked implicitly (sometimes called the **default constructor**) is the one that has no parameters.

We haven't delivered such a constructor. We haven't delivered the copying constructor, either.
The only **available constructor** is not compatible with either the default constructor or the copy constructor, which makes our program incorrect.
How can we address this issue? Is there a way to instruct the compiler to use another constructor instead of the default one?

Yes, there is. The "C++" language offers us a special syntax for this and similar situations.

Objects inside objects (3)

```
class Element {
        int value;
public:
        Element(int val) { value = val; cout << "Element(" << val << ")
constructed!" << endl; }
        int Get(void) { return value; }
        void Put(int val) { value = val; }
};
class Collection {
        Element el1, el2;
public:
        Collection(void) { cout << "Collection constructed!" << endl; }
        int Get(int elno) { return elno == 1 ? el1.Get() : el2.Get(); }
        int Put(int elno, int val) { if(elno == 1) el1.Put(val); else el2.Put(val); }
};
int main(void) {
        Collection coll;
        return 0;
```

Objects inside objects (4)

If we want **a constructor other than the default** one to be invoked during the creation of an object that is part of another object, we should use the syntax below.

Take a look at this: we can present it in the following schematic way:

Class(…) : inner_field_constr1(…), inner_field_constr2(…) { … }

This means that you have to **list all the inner objects' constructors that you wish to use instead of the default constructors**.

This is expressed in the line saying:

Collection(void) : el2(2), el1(1) { … }

Objects inside objects (4)

```
class Element {
        int value;
public:
        Element(int val) {
                value = val; cout << "Element(" << val << ") constructed!" << endl;
        }
        int Get(void) { return value; }
        void Put(int val) { value = val; }
};
class Collection {
        Element el1, el2;
public:
        Collection(void) : el2(2), el1(1) { cout << "Collection constructed!" << endl; }
        int Get(int elno) { return elno == 1 ? el1.Get() : el2.Get(); }
        int Put(int elno, int val) { if(elno == 1) el1.Put(val); else el2.Put(val); }
};
int main(void) {
        Collection coll;
        return 0;
        }
```

Objects inside objects (4)

The program may be compiled successfully and when run it produces the following output:

Element(1) constructed!
Element(2) constructed!
Collection constructed!

Note that the inner constructors have been invoked in **the sequence reflecting the order of the declaration**inside the *Collections* class (*el1* first), not in the order in which the constructors were listed in the *Class* constructor header (*el2* first).

Objects inside objects (4)

And by the way, there's the following alternation for this case: **when the constructor is divided between the declaration and the definition**, the list of alternative constructors should be **associated with the definition**, not the declaration.

This means that the following snippet is correct:

```
class X {
    public:
    X(int x) { };
};
class Y {
    X x;
    public:
    Y(int x);
};
Y::Y(int x) : x(1) { };
```

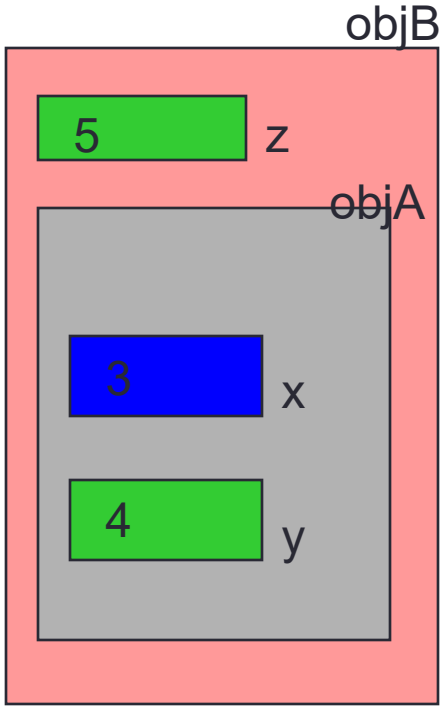
Composition (continued)

Member-objects of a class are constructed:

- In the order they are declared
- Not in the order they are listed in the constructor's member initialization list
- Before the enclosing class objects are constructed

Example

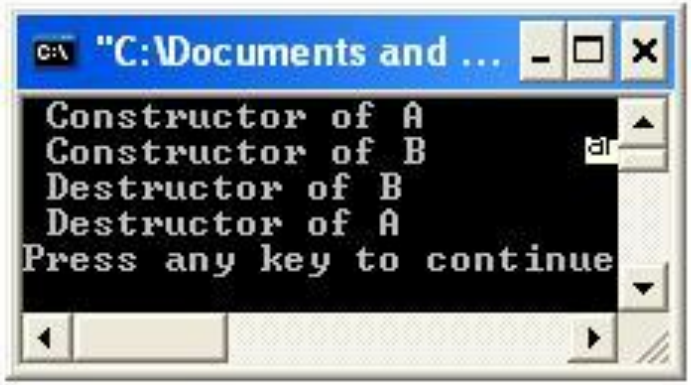
```
class A
{
public:
    int x;
    A (int a, int b) {x=a; y=b;}
    void print() {cout<<x<<"    "<<y<<endl;}
private:
    int y;
};

class B
{
public:
    A objA;
    B(int a,int b,int c): objA(a,b){z=c;}
    void print() {objA.print(); cout<<z<<endl;}
private:
    int z;
};

void main()
{
    B objB(3,4,5);
    objB.print();
    objB.objA.print();
}
```



Initialization list goes with the constructor contacting composition in order to pass parameters to the other class constructor.

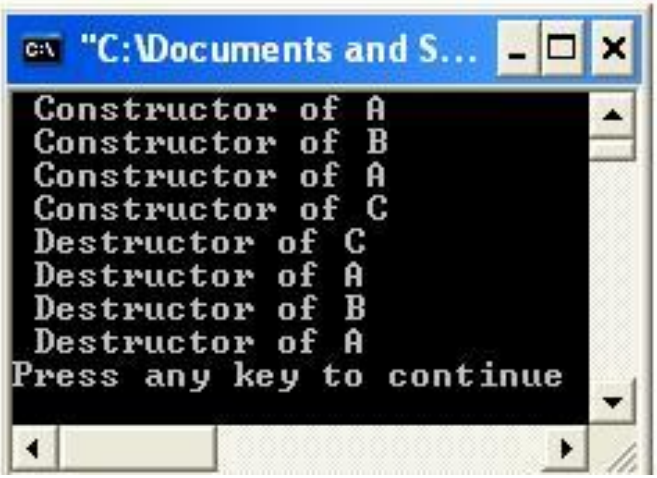
Composition (continued)

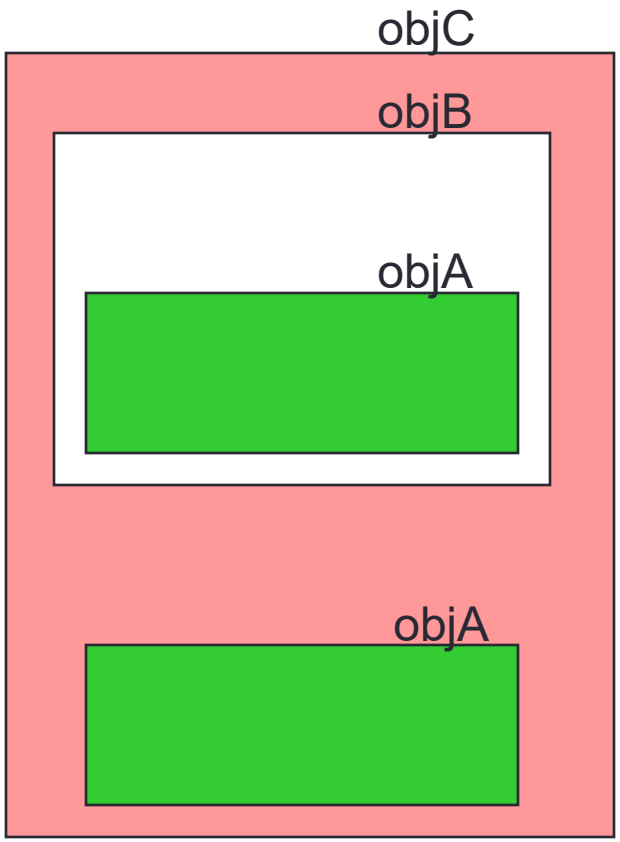
```
class A
{
public:
    A () { cout<<" Constructor of A" <<endl;}
    ~A() { cout<<" Destructor of A" <<endl;}
};

class B
{
public:
    A objA;
    B() { cout<<" Constructor of B" <<endl;}
    ~B() { cout<<" Destructor of B" <<endl;}
};

void main()
{
    B objB;
}
```

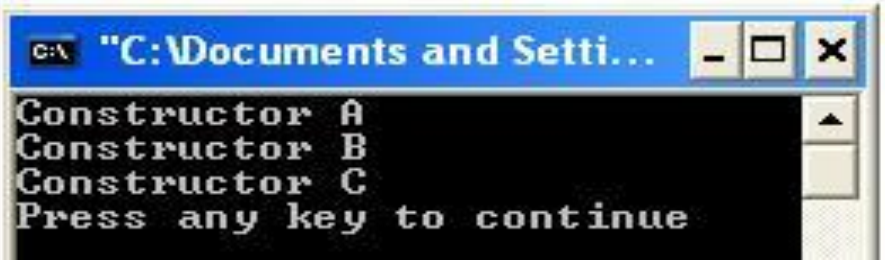


Example

```
class A
{
public:
    A () { cout<<" Constructor of A" <<endl;}
    ~A() { cout<<" Destructor of A" <<endl;}
};

class B
{
public:
    A objA;
    B() { cout<<" Constructor of B" <<endl;}
    ~B() { cout<<" Destructor of B" <<endl;}
};

class C
{
public:
    B objB;
    A objA;
    C() { cout<<" Constructor of C" <<endl;}
    ~C() { cout<<" Destructor of C" <<endl;}
};

void main()
{
    C objC;
}
```





Example

```
class A
{
public:
    A() {cout<<"Constructor A"<<endl;}
};

class B
{
public:
    B() {cout<<"Constructor B"<<endl;}
};

class C
{
public:
    A objA;
    B objB;
    C():objB(),objA() {cout<<"Constructor C"<<endl;}
};

void main()
{
    C var;
}
```